

In-Class Task 5: Keys, 2NF/3NF & ER-to-Tables

Database Systems · Related to Laboratory Work (Relational Model & Keys)

Duration: 15 minutes **Format:** Individual or pairs **Materials:** Lecture notes only

Name: Grudinskii Dmitrii Student ID: 258031192 Group: Wed 16:00

Part A: Superkey or Candidate Key? (4 minutes)

A university stores students in the following relation:

Student(StudentID, IIN, Email, FullName, GroupCode)

Business rules

- Every student has a unique StudentID, a unique IIN, and a unique university email.
- Two students can have the same full name, and many students belong to the same group.

Tasks

1. For each attribute set in the table below, write **Yes** or **No** in both columns.

Attribute set	Superkey?	Candidate key?
{StudentID}	Yes	Yes
{StudentID, FullName}	Yes	No
{FullName}	No	No
{IIN, Email}	Yes	No
{Email}	Yes	Yes
{FullName, GroupCode}	No	No

2. Pick one set that is a superkey but **not** a candidate key, and explain why in one sentence.

Second set is superkey because it uniquely identifies a row but it is not a candidate key because it has redundant data

Part B: From 1NF to 3NF (6 minutes)

An online shop stores order lines in one table. The primary key is OrderID, ProductID.

OrderItem(OrderID, ProductID, OrderDate, CustomerID, CustomerName, ProductName, UnitPrice, Quantity)

Business rules

- Each order is placed by one customer on one date.
- Each customer has one name. Each product has one name and one price.
- The quantity is recorded for each product in each order.

Sample data

OrderID	ProductID	OrderDate	CustomerID	CustomerName	ProductName	UnitPrice	Quantity
O1	P10	2026-09-14	C01	Dana	Keyboard	12000	1
O1	P20	2026-09-14	C01	Dana	Mouse	5000	2
O2	P10	2026-09-15	C02	Arman	Keyboard	12000	3

Tasks

- Complete the table below: does each attribute depend on **OrderID**, **ProductID**, or **both**?

Attribute	Depends on
OrderDate	O = OrderID
CustomerID	P = ProductID
CustomerName	O
ProductName	P
UnitPrice	O
Quantity	O

- Which of these dependencies are **partial**? Why does that break 2NF? *Every dependency should have all of the primary key not just a part of it*
- Find one **transitive** dependency. *Hint: look at the customer attributes.*
- Using the sample data, give one example of an **update anomaly**. *Updating Customer name, it should update everywhere or data becomes inconsistent*
- Decompose the table into 3NF. *Hint: you should get four tables.* Underline primary keys and mark foreign keys. [On the next page](#)

Part C: ER Diagram to Tables (5 minutes)

Scenario: Fitness Center. The ER model below is already designed.

Entities

- Member:** MemberID (key), Name, Phone.
- Trainer:** TrainerID (key), Name, Specialty.
- Class:** ClassID (key), Title, Weekday, StartTime.

Relationships

- Leads:** Trainer 1 : N Class (each class is led by exactly one trainer).
- Attends:** Member M : N Class, with attribute JoinDate.

Tasks

- How many tables do you need? Name them. [4, Member, Trainer, Class, Attends](#)
- Write the schema of each table. Underline primary keys and mark foreign keys. [On the next page](#)
- For **Leads**, which table gets the foreign key? Explain why it does not need its own table. [On the next page](#)
- For **Attends**, what is the primary key of its table, and where does JoinDate go? [On the next page](#)

Part B. Task 5

OrderItem(OrderID(PK), OrderDate, ProductID(FK), CustomerID(FK), UnitPrice, Quantity) (UnitPrice is kept because price can change after the order)

Customer(CustomerID(PK), CustomerName)

Product(ProductID(PK), Price)

Part C. Task 2

Member(MemberID(PK), Name, Phone)

Trainer(TrainerID(PK), Name, Specialty)

Class(ClassID(PK), Title, Weekday, StartTime, TrainerID(FK))

Attends(MemberID(FK, PK), ClassID(FK, PK), joinDate(PK))

Part C. Task 3

For relationship "Leads" the table "Class" gets a foreign key "TrainerID", the relationship does not need its own table because the relationship is 1 to N, which means we can safely store the 1 part in already existing table

Part C. Task 4

For relationship "Attends" the primary key of the table is (MemberID, ClassID, joinDate) which is the entire table